
Task W01: Recursion Basics

Week 1 (Base Case)

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

1 Important Guidelines

- Exam questions and problems may extend beyond tasks and lab assignments.
- No Generative AI: You must complete this assignment using only the resources provided in the course (book, lectures, and materials). Credit will not be granted for using external features or techniques not covered in this course. (See the one exception in the Applied Problems section.)
- Ask Me Questions: If you need clarification, please ask me directly. Other instructors or advisors may not be aware of the specific details and expectations of this assignment.
- The assignment instructions do not detail every step; therefore, you must read, interpret, and possibly make assumptions. If something is unclear, write down the assumption you make, proceed with it, and complete the problem. Do not leave a problem unfinished because a detail was ambiguous. Not all assumptions are automatically correct, so state yours clearly enough that I can follow your reasoning.

2 Diagram Submission Instructions

You are required to create and submit diagrams for this assignment. You may choose one of the following options.

- **Hand-drawn on paper or a whiteboard.** Scan or take clear, well-lit photos. Images must be easy to read and clearly labeled with problem numbers. This is often quicker than using diagram tools.
- **Hand-drawn using an app** (for example Vittle Pro or Adobe Fresco). Save as an image file (PNG, JPG) or PDF. This is a convenient option but may be harder without a stylus.
- **Created with diagramming software** (for example Draw.io, LibreOffice Draw, or Excalidraw). Save in the tool's native file format; do not export as an image or PDF.

-
- **Other tools.** If you would like to use a different diagramming application, ask for approval first. Note that I have limited installation access on my office computer.

Place one problem per tab. Include your name and the problem number on every diagram.

If you find something that needs to be debugged

If, while working through this assignment, you discover something in the problem statements that needs to be debugged, describe what you found in the "Bugs or Issues You Found" section of the answer sheet.

Where to submit

Commit your full submission, including your diagrams, your pseudocode, your answer sheet, and your AI conversation if you used the formatting exception, to a folder named `taskw01` in your git repository.

3 Recurrence Relation Problems

Both problems below are required. For each one, draw the full recursion tree for the given value of n , and record the final value the relation returns on your answer sheet.

3.1 Problem: $B(n)$

$$B(n) = \begin{cases} 1, & n \leq 2 \\ B(n-1) - B(n-2), & n > 2 \end{cases}$$

Draw the recursion tree for $B(6)$. Record the value of $B(6)$ on your answer sheet.

3.2 Problem: $D(n)$

$$D(n) = \begin{cases} 1, & n \leq 3 \\ D(n-2) + D(n-1) - D(n-3), & n > 3 \end{cases}$$

Draw the recursion tree for $D(6)$. Record the value of $D(6)$ on your answer sheet.

4 Applied Problems

Four problems are described below, in no particular order of difficulty. Read through all four before you start.

If you have not written pseudocode before, or would like to see it written the way Cormen, Leiserson, Rivest, and Stein (CLRS) and Carrano's Walls and Mirrors write it, this guide covers those conventions with worked examples: https://course.ccs.neu.edu/cs3000/resources/latex_pseudocode.pdf. You do not need to use the LaTeX package described in that guide, or match its typesetting exactly; plain pseudocode that follows the same conventions (numbered lines, indentation, real `for/while/if` structure) is fine. You may also write your solution as working code in Java or C/C++ instead of pseudocode, if you would rather work that way.

Once you have solved a problem, and only after you have solved it, you may use an AI tool solely to format your finished pseudocode into the conventions above. If you do this, you must explicitly tell the AI not to change or fix your algorithm, only to format it, and you must submit the full AI conversation alongside your pseudocode. This exception does not extend to using AI to

solve the problem, or to fix a mistake in your algorithm; that is still covered by the No Generative AI guideline above.

- Every solution must use recursion. A solution that does not use recursion will not receive credit, even if it produces the correct result.
- Choose **one** of the four and complete it in full. Write correct recursive pseudocode, or working recursive code in Java or C/C++, for a solution, and draw a full diagram tracing your solution on the example input given. A box trace or a recursion tree is acceptable, whichever fits your solution. Record the final returned value on your answer sheet.
- For the **other three**, write your best attempt at a recursive solution, in pseudocode or in Java or C/C++. No diagram is required for these three.

4.1 Problem 1

Write a method that takes a positive integer and returns the sum of its digits. For example, the digit sum of 48173 is $4 + 8 + 1 + 7 + 3 = 23$.

Example input if you choose this one: 48173.

4.2 Problem 2

A staircase has n steps. You can climb it by taking either 1 step or 2 steps at a time. Write a method that returns the number of different ways to climb a staircase with n steps. A recursion tree is an acceptable diagram for this problem, if a box trace does not fit your solution.

Example input if you choose this one: 6 steps.

4.3 Problem 3

Write a method that takes a string of characters and returns true if the string reads the same forwards and backwards, and false otherwise.

Example input if you choose this one: "racecar".

4.4 Problem 4

Write a method that returns the greatest common divisor of two positive integers a and b , using Euclid's algorithm: the greatest common divisor of a and b is the same as the greatest common divisor of b and the remainder of a divided by b , and the greatest common divisor of any number and 0 is that number.

Example input if you choose this one: $a = 150$, $b = 84$.